

-SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INTERACTIVE TREE SIMULATOR

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	Assessment:	Assessment Tool 1 – Interactive Tree Simulator
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	<i>Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.</i>		
Student Name:	S.Roshni	Reg. No.:	192525185
Section / Batch:	AI&ML	Date:	29/07/2026

Code of Conduct

I S.Roshni(192525185)_____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Roshni_____

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic	Focus	Q Nos	Marks
Binary Search Tree	Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals	Traversal types, In order, Post order and Pre order	2	20
AVL Tree	Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree	Properties, structures and Operations	4	20
Red Black Tree	Properties, Example and Operations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Interactive Tree Simulator

- Construct a Binary Search Tree (BST) by inserting the following keys: 50, 30, 70, 20, 40, 60, 80
Perform:
 - Inorder Traversal
 - Search for 60
 - Delete 30
 - Display the final BST.
- Complete Tree Traversals by Inserting 50, 30, 70, 20, 40, 60, 80. Find all the traversals.
 - Inorder
 - Preorder
 - Postorder
- Insert 70, 50, 90, 40, 60, 80, 100, 55 into an AVL Tree. Delete 40 and 90. Analyze how the tree rebalances after each deletion.
- Construct a B-Tree of order 3 by inserting 45, 25, 65, 15, 35, 55, 75
- Insert 100, 80, 120, 60, 90, 110, 130 into a Red-Black Tree. Analyze their heights.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations
Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, node split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Construct a Binary Search Tree (BST) by inserting the following keys: 50, 30, 70, 20, 40, 60, 80 Perform:

a. Inorder Traversal

The screenshot shows the 'Tree Traversals' web application. On the left, the 'Input' field contains '50,30,70,20,40,60,80'. Below it, the 'Traversal Order' section has buttons for 'Pre', 'In', and 'Post', with 'In' selected. There are also buttons for 'Generate Default', 'Visualize', and 'Start Simulation'. On the right, a partial binary search tree is visible with a root node '50' and its left child '20'. A modal dialog box in the center displays the 'Traversal result: [20, 30, 40, 50, 60, 70, 80]' with an 'OKAY' button.

b. Search for 6

The screenshot shows the 'Depth First Search' web application. On the left, the 'Input' field contains '50,30,70,20,40,60,80' and the 'Target Value' field contains '60'. There are buttons for 'Generate Default', 'Visualize', and 'Start Simulation'. On the right, a partial binary search tree is visible with a root node '50' and its left child '20'. A modal dialog box in the center displays the message 'Found target 60 at node root.right.left.' with an 'OKAY' button.

Input
Input is level-by-level order.

50,30,70,20,40,60,80

Enter numbers separated by commas. Use 'null' for empty nodes.

Target Value

60

 **Generate Default**

 **Visualize**

 **Start Simulation**

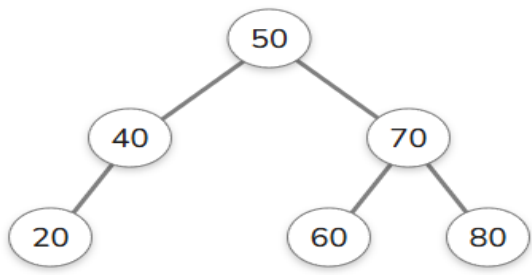


```
graph TD; 50((50)) --> 30((30)); 50 --> 70((70)); 30 --> 20((20)); 30 --> 40((40)); 70 --> 60((60)); 70 --> 80((80));
```

C.Delete 30

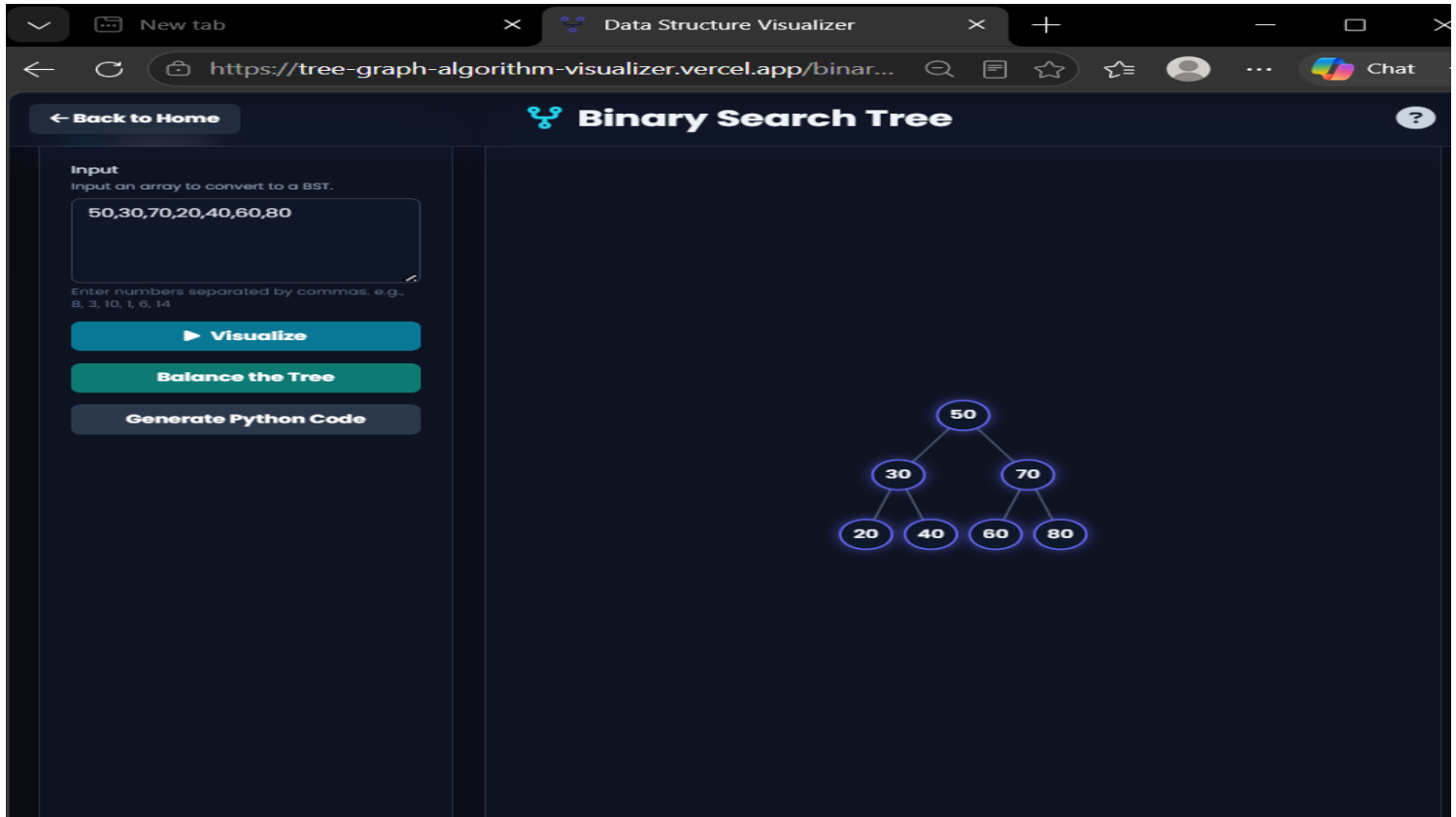
`insert(node.right, key)`

Enter a number: 31  **INSERT** **DELETE** **CLEAR**     



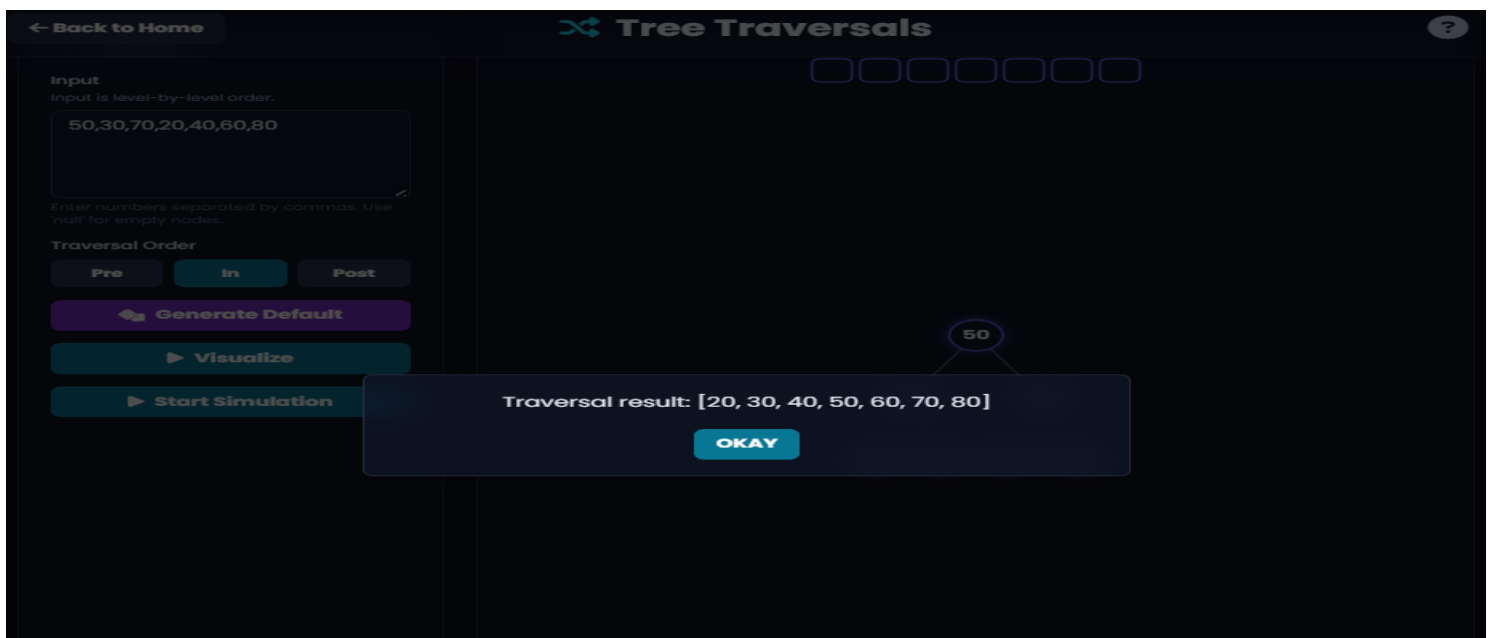
```
graph TD; 50((50)) --> 40((40)); 50 --> 70((70)); 40 --> 20((20)); 70 --> 60((60)); 70 --> 80((80));
```

d.Display the final BST.



2.Complete Tree Traversals by Inserting 50, 30, 70, 20, 40, 60, 80. Find all the traversals.

a.Inorder



b.Preorder

[← Back to Home](#)

Tree Traversals

?

Input

Input is level-by-level order.

50,30,70,20,40,60,80

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

50

30

20

40

70

60

80

50

Traversal result: [50, 30, 20, 40, 70, 60, 80]

OKAY

c.Postorder

[← Back to Home](#)

Tree Traversals

?

Input

Input is level-by-level order.

50,30,70,20,40,60,80

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

20 40 30 60 80 70 50

50

Traversal result: [20, 40, 30, 60, 80, 70, 50]

OKAY

3.Insert 70, 50, 90, 40, 60, 80, 100, 55 into an AVL Tree. Delete 40 and 90. Analyze how the tree rebalances after each deletion.

seudocode

```

function rebalance(node):
    updateHeight(node)
    nodeBf = balanceFactor(node)
    if nodeBf > 1:
        if balanceFactor(node.left) >= 0:
            rotateRight(node)
        else:
            rotateLeft(node.left)
            rotateRight(node)
    if nodeBf < -1:
        if balanceFactor(node.right) <= 0:
            rotateLeft(node)
        else:
            rotateRight(node.right)
            
```

Enter a number: 24

▶

INSERT

DELETE

CLEAR

↶

↷

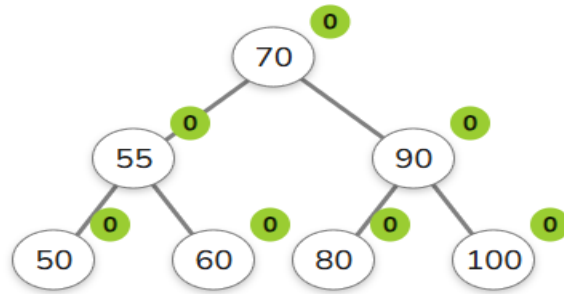
📁

🔗

```

graph TD
    70((70)) --- 50((50))
    70 --- 90((90))
    50 --- 40((40))
    50 --- 60((60))
    60 --- 55((55))
    90 --- 80((80))
    90 --- 100((100))
    
```

Delete 40:



Delete 90:

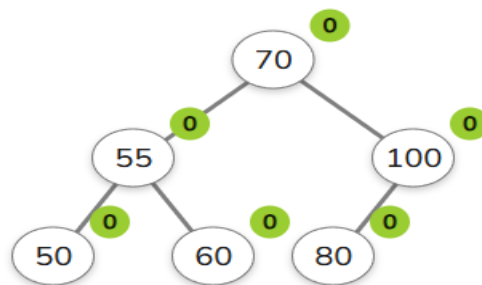
Enter a number: 98



INSERT

DELETE

CLEAR



4. Construct a B-Tree of order 3 by inserting 45, 25, 65, 15, 35, 55, 75

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), each node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

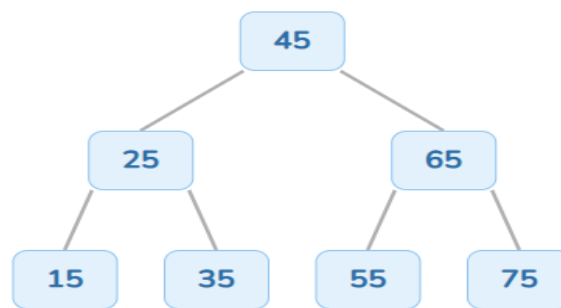
Enter a number:



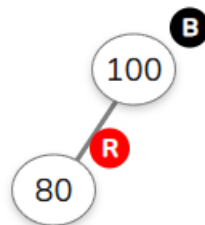
INSERT

SEARCH

CLEAR



5. Insert 100, 80, 120, 60, 90, 110, 130 into a Red-Black Tree. Analyze their heights.



Enter a number:



INSERT

CLEAR

